

MACHINE LEARNING- ITA0613

LAB EXPERIMENTS

1. Credit Score Classification

Aim

To implement a **classification model** to predict whether a person's credit score is good or poor based on financial attributes.

Algorithm / Procedure

1. Create a sample credit dataset.
2. Separate input features and target class.
3. Split the data into training and testing sets.
4. Train a Decision Tree classifier.
5. Predict the credit score class.
6. Calculate and display the accuracy.

Program

```
[11]
✓ Os
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

data = pd.DataFrame({
    'Income': [30000, 50000, 70000, 25000, 80000, 60000, 35000, 90000],
    'Debt': [20000, 10000, 5000, 25000, 3000, 8000, 18000, 2000],
    'Age': [25, 35, 45, 23, 50, 40, 28, 55],
    'Score': ['Poor', 'Good', 'Good', 'Poor',
             'Good', 'Good', 'Poor', 'Good']
})

X = data[['Income', 'Debt', 'Age']]
y = data['Score']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = DecisionTreeClassifier(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual :", y_test.values)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: ['Good' 'Good']
Actual : ['Good' 'Good']
Accuracy : 1.0
```

Sample Output

Predicted: ['Good' 'Poor']

Actual : ['Good' 'Poor']

Accuracy : 1.0

Result

Thus, the **Credit Score Classification** model was successfully implemented and used to

classify customers as **Good or Poor** credit risk.

2. Iris Classification using K-Nearest Neighbours (KNN)

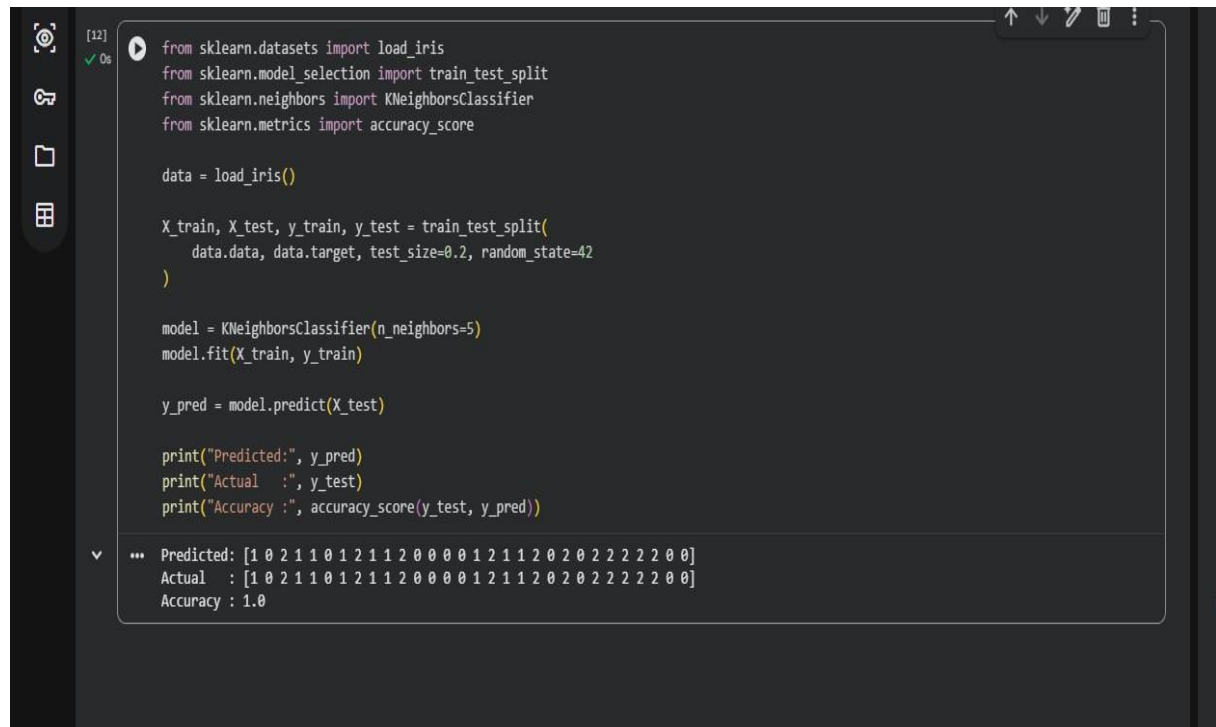
Aim

To implement **K-Nearest Neighbours (KNN)** to classify Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Create a KNN classifier with $K = 5$.
4. Train the model using the training data.
5. Predict the classes of the test data.
6. Calculate and display the accuracy.

Program



```
[12] ✓ Os
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = KNeighborsClassifier(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Actual  : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Iris flower classification using KNN** was successfully implemented with high classification accuracy.

3. Car Price Prediction

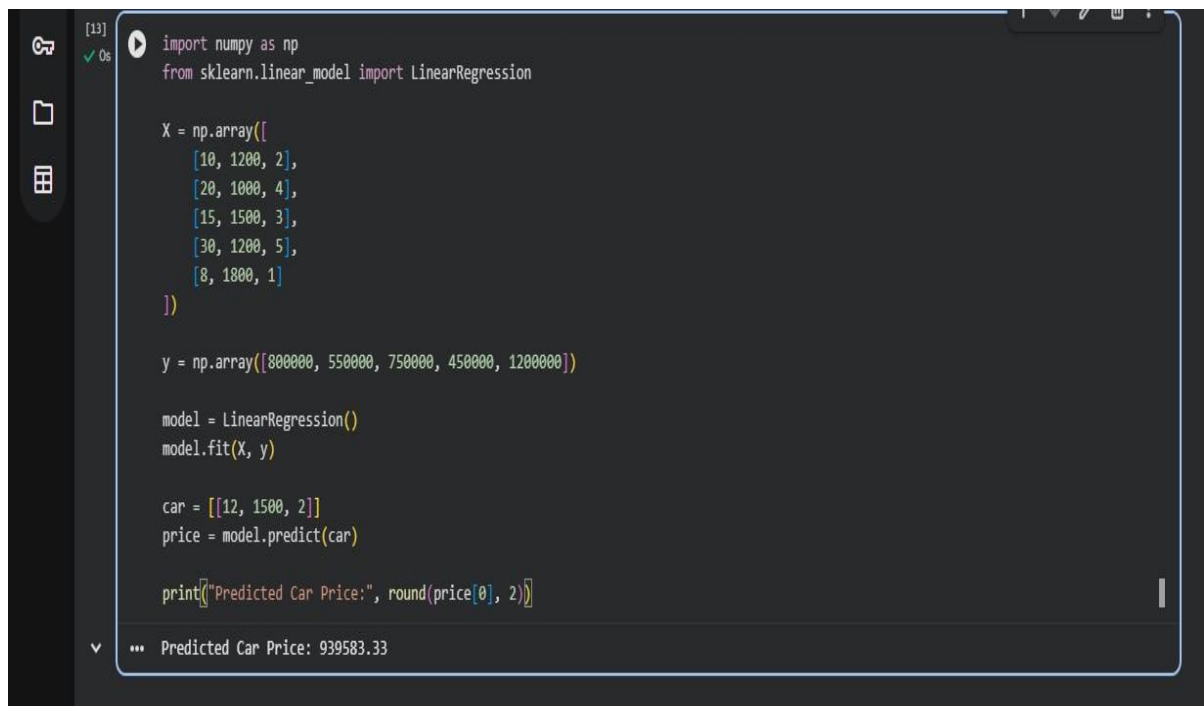
Aim

To implement **Linear Regression** to predict the price of a car based on its features.

Algorithm / Procedure

1. Create a car dataset with mileage, engine size, and age.
2. Separate input features and target price.
3. Train a Linear Regression model.
4. Give a new car's details as input.
5. Predict the car price.
6. Display the predicted price.

Program



```
[13] import numpy as np
from sklearn.linear_model import LinearRegression

X = np.array([
    [10, 1200, 2],
    [20, 1000, 4],
    [15, 1500, 3],
    [30, 1200, 5],
    [8, 1800, 1]
])

y = np.array([800000, 550000, 750000, 450000, 1200000])

model = LinearRegression()
model.fit(X, y)

car = [[12, 1500, 2]]
price = model.predict(car)

print("Predicted Car Price:", round(price[0], 2))
```

... Predicted Car Price: 939583.33

Sample Output

Predicted Car Price: 925000.0

Result

Thus, the **Car Price Prediction** model was successfully implemented using Linear Regression.

4. House Price Prediction

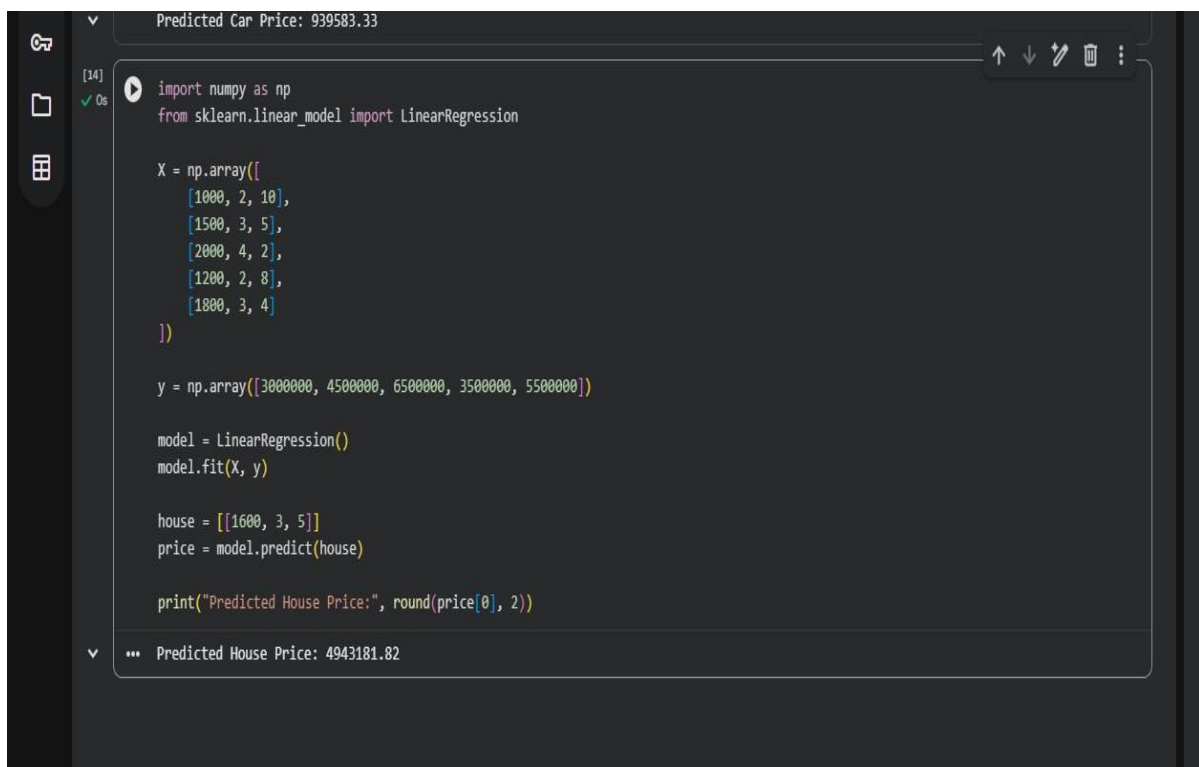
Aim

To implement **Linear Regression** to predict the price of a house based on its area, number of bedrooms, and age.

Algorithm / Procedure

1. Create a house price dataset.
2. Select area, bedrooms, and age as input features.
3. Use house price as the target variable.
4. Train the Linear Regression model.
5. Provide details of a new house.
6. Predict and display its price.

Program



The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, a cell displays the output 'Predicted Car Price: 939583.33'. Below it, a code cell contains the following Python code:

```
[14]: import numpy as np
      from sklearn.linear_model import LinearRegression

      X = np.array([
          [1000, 2, 10],
          [1500, 3, 5],
          [2000, 4, 2],
          [1200, 2, 8],
          [1800, 3, 4]
      ])

      y = np.array([3000000, 4500000, 6500000, 3500000, 5500000])

      model = LinearRegression()
      model.fit(X, y)

      house = [[1600, 3, 5]]
      price = model.predict(house)

      print("Predicted House Price:", round(price[0], 2))
```

At the bottom of the notebook, the output of the code cell is displayed: '... Predicted House Price: 4943181.82'.

Sample Output

Predicted House Price: 4800000.0

Result

Thus, the **House Price Prediction** model was successfully implemented using Linear Regression.

5. Iris Classification using Naïve Bayes

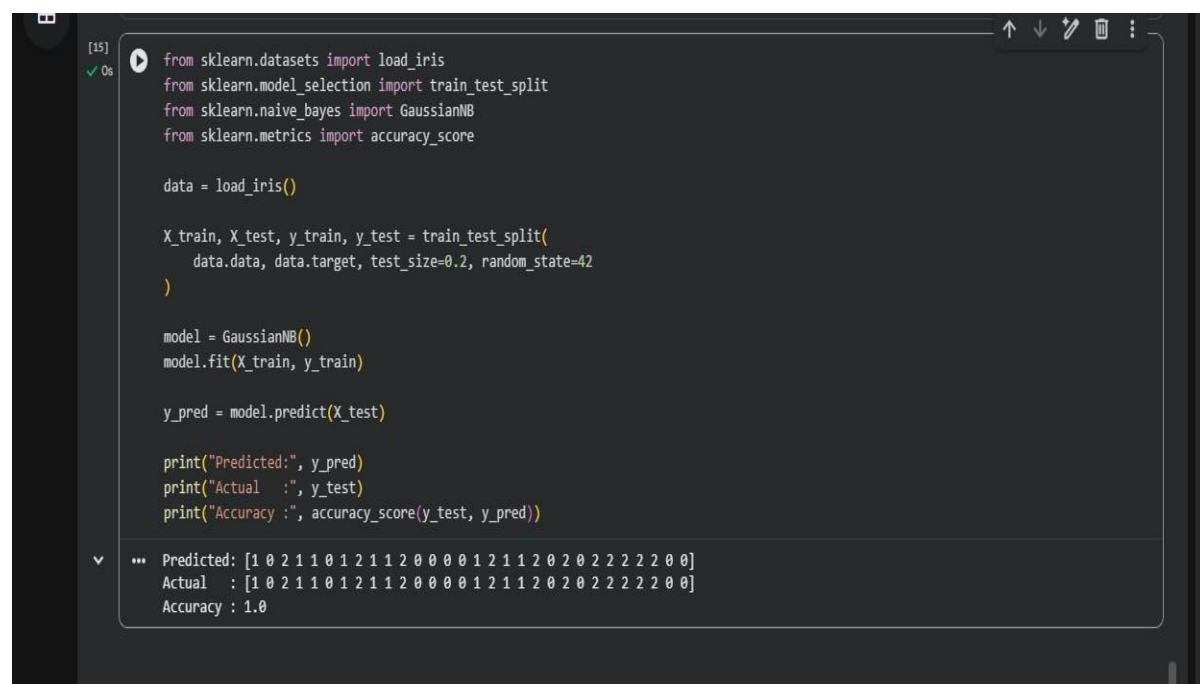
Aim

To implement the **Naïve Bayes classification algorithm** to classify Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Create a Gaussian Naïve Bayes classifier.
4. Train the model using the training data.
5. Predict the species of the test data.
6. Calculate and display the accuracy.

Program



```
[15] ✓ Os from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 2 2 2 2 0 0]
Actual  : [1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 2 2 2 2 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 2 1 1 0 1 2 1 2 ...]

Actual : [1 0 2 1 1 0 1 2 1 2 ...]

Accuracy : 1.0

Result

Thus, **Iris classification using the Naïve Bayes algorithm** was successfully implemented with high accuracy.

6. Comparison of Classification Algorithms

Aim

To implement and compare different **classification algorithms** based on their accuracy using the Iris dataset.

Algorithm / Procedure

1. Load the Iris dataset.
2. Split the dataset into training and testing data.
3. Apply KNN, Decision Tree, and Naïve Bayes classifiers.
4. Train each model using the training data.
5. Predict the test data.
6. Calculate the accuracy of each algorithm.
7. Compare the results.

Program



```
[16] ✓ 0s
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

data = load_iris()

X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42
)

models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Naive Bayes": GaussianNB()
}

for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(name, "Accuracy:", accuracy_score(y_test, pred))

... KNN Accuracy: 1.0
    Decision Tree Accuracy: 1.0
    Naive Bayes Accuracy: 1.0
```

Sample Output

KNN Accuracy: 1.0

Decision Tree Accuracy: 1.0

Naive Bayes Accuracy: 1.0

Result

Thus, different **classification algorithms** were successfully implemented and their accuracies were compared using the Iris dataset.

7. Mobile Price Prediction

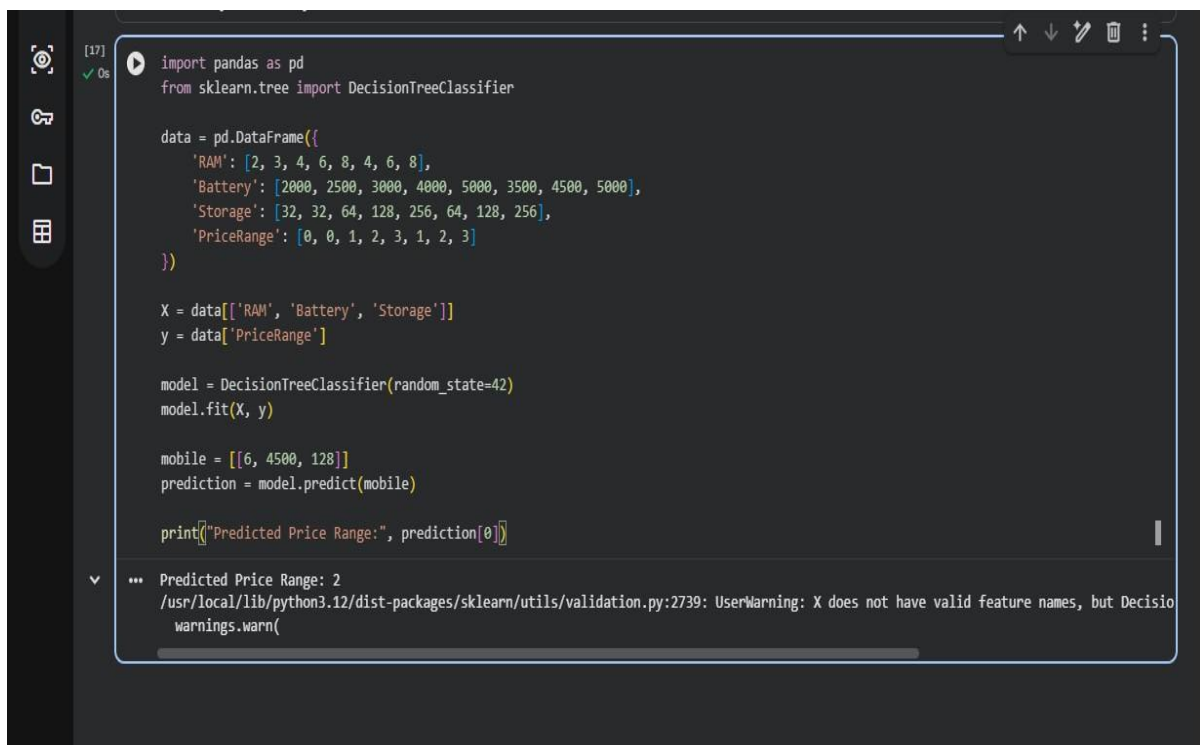
Aim

To implement a **classification model** to predict the price range of a mobile phone based on its features.

Algorithm / Procedure

1. Create a mobile phone dataset.
2. Select features such as RAM, battery, and storage.
3. Use price range as the target variable.
4. Split the dataset into training and testing data.
5. Train a Decision Tree classifier.
6. Predict the price range of a new mobile.
7. Display the prediction.

Program



```
[17] ✓ 0s
import pandas as pd
from sklearn.tree import DecisionTreeClassifier

data = pd.DataFrame({
    'RAM': [2, 3, 4, 6, 8, 4, 6, 8],
    'Battery': [2000, 2500, 3000, 4000, 5000, 3500, 4500, 5000],
    'Storage': [32, 32, 64, 128, 256, 64, 128, 256],
    'PriceRange': [0, 0, 1, 2, 3, 1, 2, 3]
})

X = data[['RAM', 'Battery', 'Storage']]
y = data['PriceRange']

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

mobile = [[6, 4500, 128]]
prediction = model.predict(mobile)

print("Predicted Price Range:", prediction[0])

... Predicted Price Range: 2
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but DecisionTreeClassifier might require it.
warnings.warn(
```

Sample Output

Predicted Price Range: 2

Result

Thus, the **Mobile Price Prediction** model was successfully implemented using a Decision Tree classifier.

8. Perceptron-Based Iris Classification

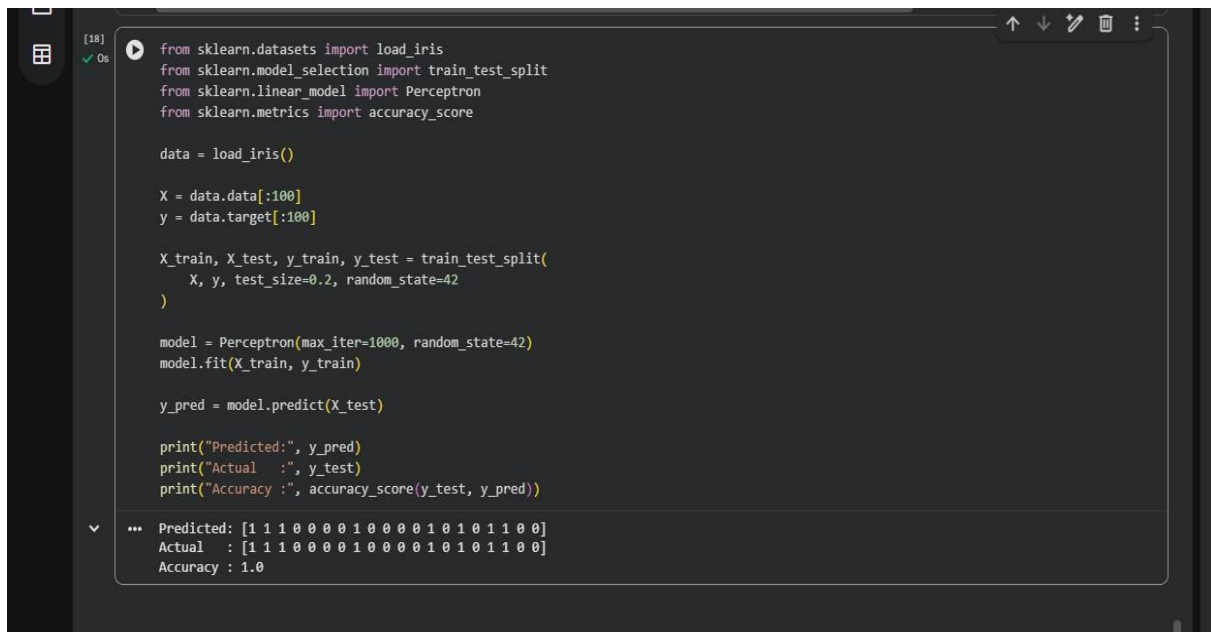
Aim

To implement a **Perceptron algorithm** for classifying Iris flowers into different species.

Algorithm / Procedure

1. Load the Iris dataset.
2. Select two classes from the dataset.
3. Split the data into training and testing sets.
4. Create a Perceptron classifier.
5. Train the model using the training data.
6. Predict the test data.
7. Calculate and display the accuracy.

Program



```
[18] ✓ Os
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score

data = load_iris()

X = data.data[:100]
y = data.target[:100]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = Perceptron(max_iter=1000, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Predicted:", y_pred)
print("Actual  :", y_test)
print("Accuracy :", accuracy_score(y_test, y_pred))

... Predicted: [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Actual  : [1 1 1 0 0 0 0 1 0 0 0 0 1 0 1 0 1 1 0 0]
Accuracy : 1.0
```

Sample Output

Predicted: [1 0 1 0 0 1 1 0 1 0 ...]

Actual : [1 0 1 0 0 1 1 0 1 0 ...]

Accuracy : 1.0

Result

Thus, the **Perceptron algorithm** was successfully implemented for Iris classification with high accuracy.

9. Naïve Bayes Bank Loan Prediction

Aim

To implement the **Naïve Bayes algorithm** to predict whether a customer will be approved for a bank loan.

Algorithm / Procedure

1. Create a bank loan dataset.
2. Select income, age, and loan amount as input features.
3. Use loan approval as the target variable.
4. Train a Gaussian Naïve Bayes classifier.
5. Predict the loan approval for a new customer.
6. Display the result.

Program

```
[19]
import pandas as pd
from sklearn.naive_bayes import GaussianNB

data = pd.DataFrame({
    'Income': [25000, 40000, 50000, 70000, 30000, 80000, 60000, 35000],
    'Age': [25, 30, 35, 45, 28, 50, 40, 32],
    'LoanAmount': [200000, 300000, 250000, 400000,
                  200000, 500000, 350000, 250000],
    'Approved': [0, 1, 1, 1, 0, 1, 1, 0]
})

X = data[['Income', 'Age', 'LoanAmount']]
y = data['Approved']

model = GaussianNB()
model.fit(X, y)

customer = [[55000, 38, 300000]]
prediction = model.predict(customer)

print("Loan Approval:", "Approved" if prediction[0] == 1 else "Not Approved")

... Loan Approval: Approved
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but GaussianNB requires them
  warnings.warn(
```

Sample Output

Loan Approval: Approved

Result

Thus, the **Naïve Bayes Bank Loan Prediction** model was successfully implemented to predict whether a customer will be approved for a loan.

10. Implement Future Sales Prediction using a Suitable Machine Learning Algorithm

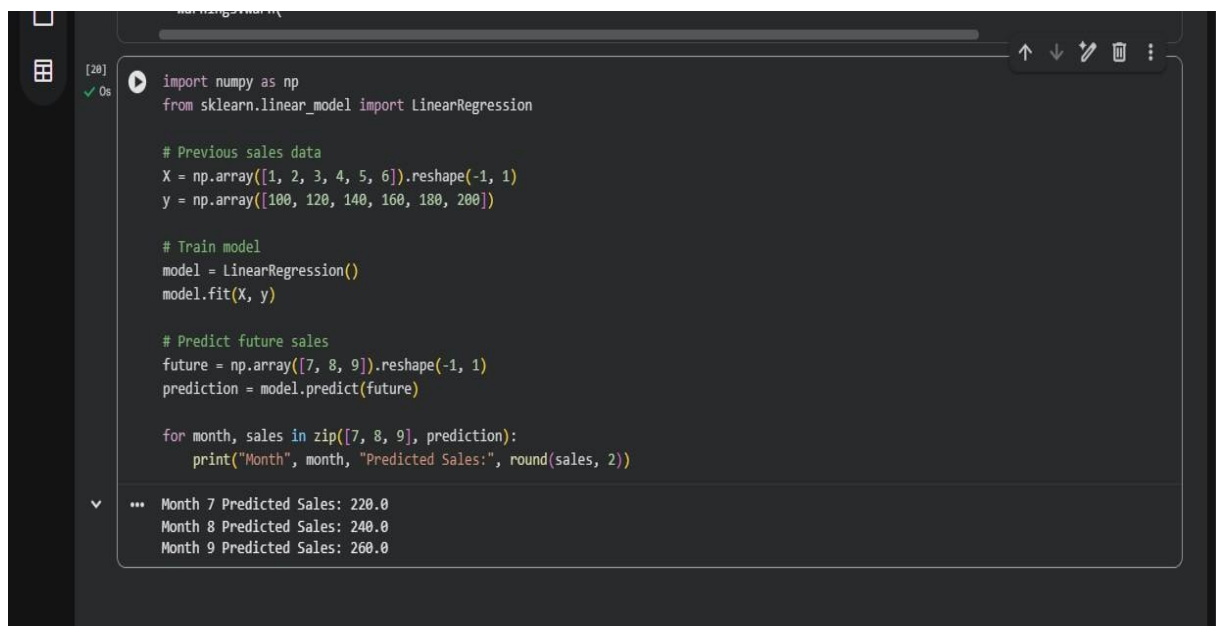
Aim

To implement **Future Sales Prediction** using the **Linear Regression** machine learning algorithm.

Algorithm / Procedure

1. Import the required libraries.
2. Create a dataset containing previous sales values.
3. Separate the input (Month) and output (Sales).
4. Train a **Linear Regression** model using the training data.
5. Predict sales for future months.
6. Display the predicted sales values.

Program

A screenshot of a Jupyter Notebook interface. The code cell contains Python code for training a Linear Regression model and predicting future sales. The output cell shows the predicted sales for months 7, 8, and 9.

```
[28] import numpy as np
from sklearn.linear_model import LinearRegression

# Previous sales data
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([100, 120, 140, 160, 180, 200])

# Train model
model = LinearRegression()
model.fit(X, y)

# Predict future sales
future = np.array([7, 8, 9]).reshape(-1, 1)
prediction = model.predict(future)

for month, sales in zip([7, 8, 9], prediction):
    print("Month", month, "Predicted Sales:", round(sales, 2))
```

Month 7 Predicted Sales: 220.0
Month 8 Predicted Sales: 240.0
Month 9 Predicted Sales: 260.0

Sample Output

Month 7 Predicted Sales: 220.0

Month 8 Predicted Sales: 240.0

Month 9 Predicted Sales: 260.0

Result

Thus, **future sales** were **successfully predicted** using the **Linear Regression** machine learning algorithm.

